Autoclaim Registry Audit

Côme du Crest

2025-03-24

Table of contents

- Table of contents
- Autoclaim Registry Audit
  - Scope
  - Context
  - Status
- Issues
  - [Medium] Batch claims will not work with claim action
  - [Low] ClaimActionUpgradeable may give too much power to the owner
  - [Info] Balancer swap vulnerable to multi-block sandwich attacks
  - [Info] Curve swap sandwich protection assumes USD peg of DAI
  - Optimisations and miscellaneous

Autoclaim Registry Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes all contracts within `gnosischain/autoclaim-registry` as of commit `0x0fb58ff`.

Context

The repository contains a `ClaimRegistry` contract whose goal is to automatically claim validators GNO rewards. The automation is performed via two functions `resolve()` and `claimBatch()` called by PowerPool agents. The first function `resolve()` returns wether the contract needs to be called and the calldata to send (`claimBatch.selector` and validators addresses). The second function claims rewards for the registered validators after a deadline or after a certain amount of rewards has been accumulated.

For each validator an action contract can be called after the claim. The only available action contract right now attempts to swap the GNO rewards to EURe by first swapping them to xDAI using Balancer and then to EURe using Curve.

The main attack vectors for the contract are sandwich attacks on the Balancer and Curve swaps. Overall the safety of the contract can be assessed by the user's sensitivity to these attacks as well as the amount of funds at risk. Two issues already reported for a previous audit of `GnoClaimSwapForward` have been reported again as `Info` level as they simply highlight risks that may be accepted by certain users.

Status

The report has been sent to the core developer.

The issues have been fixed in PR `gnosischain/autoclaim-registry/pull/2`. The last commit looked at is `0x9b550cf`. No issues remain.

Issues

[Medium] Batch claims will not work with claim action

Summary

Validator rewards are meant to be claimed in batches via `claimBatch()`. However, the sandwich protection mechanism of `ClaimActionUpgradeable.balancerSwapGnoToWxdai()` reverts if the underlying balancer pool has been touched in the current block. If two validators registered the swap action, they will not be able to be claimed in the same batch.

Vulnerability Detail

`claimBatch()` loops over the input withdrawal addresses and tries to claim for every address:

```
    function claimBatch(address[] calldata withdrawalAddresses) public {
        for (uint256 i = 0; i < withdrawalAddresses.length; i++) {
            try this.claim(withdrawalAddresses[i]) {} catch Error(
                string memory reason
            ) {
                emit ClaimFailed(withdrawalAddresses[i], reason);
            } catch {
                emit ClaimFailed(withdrawalAddresses[i], "Unknown error");
            }
        }
        emit ClaimBatch(msg.sender, withdrawalAddresses);
    }
```

The `claim()` function will call the post claim action contract if registered:

```
    function claim(address withdrawalAddress) public {
        ...

        address userActionContract = actionContract[withdrawalAddress];
        if (userActionContract != address(0)) {
            ...
            IClaimActionUpgradeable(userActionContract).executePostClaimAction(
                withdrawalAddress,
                amount
            );
        }
    }
```

The function `ClaimActionUpgradeable.executePostClaimAction()` will eventually check the `lastChangeBlock` of the pool and revert if swaps already occurred on the pool:

```
    function executePostClaimAction(
        address _withdrawalAddress,
```

```
        uint256 _amount
    ) external onlyClaimRegistry {
        swapAndForward(_withdrawalAddress, _amount);
    }

    ...
    function swapAndForward(address claimAddress, uint256 amount) private {
        ...
        balancerSwapGnoToWxdai(amount);
        ...
    }

    ...
    function balancerSwapGnoToWxdai(uint256 gnoAmount) private {
        ...
        if (balancerSandwichPrevention) {
            (, , uint256 lastChangeBlock, ) = vaultContract.getPoolTokenInfo(
                poolId,
                IERC20(gnoTokenAddress)
            );

            require(
                lastChangeBlock < block.number,
                "Balancer pool has been used in this block already. Revert to
                    prevent in-block sandwiching attacks."
            );
        }
        ...
    }
```

Impact

Batch claims will fail for users using this post claim action if another withdrawal address registered the same post claim action is processed earlier in the batch. Anyone can register their withdrawal addresses with this post claim action to make the automated claim process fail for others.

Code Snippets

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimActionUpgradeable.sol#L141-L151

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimRegistryUpgradeable.sol#L360-L376

Recommendation

The `ClaimActionUpgradeable` should process batch actions to swap every users' GNO in one go. This requires redesigning the registry so that when claiming a batch the action contract is called after the GNO of every user has already been claimed from the withdrawal contract with a list of the addresses to process the action on. The function would be `executePostClaimActions(address[] withdrawalAddresses, uint256[] amounts)`.

Response

This is fixed in commits `0x99e3aad` and `0x3bd9dab` by computing the price of GNO in EUR using Chainlink oracles.

[Low] ClaimActionUpgradeable may give too much power to the owner

Summary

The contract `ClaimActionUpgradeable` provides for an owner that can update the implementation pointed by the proxy and update the sandwich prevention settings. This seems like too much power given unnecessarily to the owner.

Vulnerability Detail

The owner can update the implementation through the EIP1967 proxy function `upgradeToAndCall()` which verifies that the upgrade is authorized by calling `_authorizeUpgrade()` which only enforces the call be done by the owner:

```
    function upgradeToAndCall(address newImplementation, bytes memory data)
        public payable virtual onlyProxy {
        _authorizeUpgrade(newImplementation);
        _upgradeToAndCallUUPS(newImplementation, data);
    }
```

```
    function _authorizeUpgrade(
        address newImplementation
    ) internal override onlyOwner {}
```

The owner can also update the settings of sandwich prevention mechanisms:

```
    function changeBalancerSandwichPrevention(
        bool preventSandwiching
    ) public onlyOwner {
        balancerSandwichPrevention = preventSandwiching;
    }

    ...
    function changeCurveMaxDiffSandwichPrevention(
        uint256 maxDiffValue
    ) public onlyOwner {
        curveMaxDiff = maxDiffValue;
    }
```

Impact

Through these functions, the owner of the contract can steal all the GNO tokens (and pending rewards) of validators up to the amount of GNO tokens approved to `ClaimActionUpgradeable`.

This can be done by upgrading the implementation to a simple contract that transfers every held GNO to the owner or by removing the sandwich protection and sandwich the swaps.

Code Snippets

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimActionUpgradeable.sol#L66-L68

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimActionUpgradeable.sol#L228-L240

Recommendation

Do not make this contract upgradeable so users do not have to rely on the trust in the owner not to siphon their funds.

New action contracts can always be deployed and users can decide to activate them via `ClaimRegistryUpgradeable.setActionContract()`.

Response

The claim action contract is no longer upgradeable and does not have an owner as of commit `0x388a723`.

[Info] Balancer swap vulnerable to multi-block sandwich attacks

Summary

The sandwich protection implemented for the Balancer swap checks whether the pool's balances have changed in the current block and revert when that is the case. This protection can be circumvented by an attacker that would front-run a contract interaction in the previous block and back-run it normally.

This is made easier by the non-restricted function `claim()` that can be called by anyone for any address and by the low transaction fees on Gnosis Chain.

Vulnerability Detail

The sandwich protection for the Balancer swap reverts when the pool's balances have been updated in the same block:

```
1     function balancerSwapGnoToWxdai(uint256 gnoAmount) private {
2         address vaultAddress = 0xBA12222222228d8Ba445958a75a0704d566BF2C8;
3         Balancer vaultContract = Balancer(vaultAddress);
4         bytes32 poolId = 0
              x8189c4c96826d016a99986394103dfa9ae41e7ee000200000000000000000aa;
5
6         // Poor mans in-block sandwich prevention. If the pool has been touched
               in the same block, revert.
7         // There is about 1 balancer transaction per 100 blocks, so it has a 1%
               chance to give a false positive.
8         if (balancerSandwichPrevention) {
9             (, , uint256 lastChangeBlock, ) = vaultContract.getPoolTokenInfo(
10                poolId,
11                IERC20(gnoTokenAddress)
12            );
13
14            require(
15                lastChangeBlock < block.number,
16                "Balancer pool has been used in this block already. Revert to
                      prevent in-block sandwiching attacks."
17            );
18        }
19
20        ...
21    }
```

Impact

An attacker only needs to control the last transaction in a block and the first transaction in the following block to be guaranteed to sandwich the Balancer swap. Moreover, the comment states that there is about 1 balancer transaction per 100 blocks (I assume targeting this pool), meaning that a random

cross-blocks sandwich attack has about 0.5% chance of failure (fails if the consecutive blocks contain a balancer transaction for that pool and it affects the pool in the wrong direction).

Whether this is an acceptable risk or not should be determined by the user.

Code Snippets

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimActionUpgradeable.sol#L134-L182

Recommendation

Use an off-chain oracle to get the price of GNO in EURe to correctly protect the two swaps against sandwich attacks.

Response

This is fixed in commits `0x99e3aad` and `0x3bd9dab` by computing the price of GNO in EUR using Chainlink oracles.

[Info] Curve swap sandwich protection assumes USD peg of DAI

Summary

The sandwich attack protection guaranteeing a fair price for the Curve swap assumes 1 wxDAI should be worth 1 USD which is not the long term philosophy of DAI.

Vulnerability Detail

The function `curveSwapWxdaiEure()` gets the price of `EURe`/`wxDAI` via `curveContract.price_oracle()` and uses that value to compute the minimally accepted amount of EURe received:

```
        function curveSwapWxdaiEure(uint256 wxdaiAmount) private {
        address curveAddress = 0xE3FFF29d4DC930EBb787FeCd49Ee5963DADf60b6;
        Curve curveContract = Curve(curveAddress);
        uint256 oraclePrice = curveContract.price_oracle(); // wxDAI you get
            for 1 EURe multiplied by 1e18

        uint256 minReceive = 0; // TODO: Can be sandwiched to oblivion.
        IERC20(wxdaiTokenAddress).approve(curveAddress, wxdaiAmount);
        // Pool tokens 0=EURe, 1=wxDAI, 2=USDC,3=USDT
        uint inTokenIndex = 1; // wxDAI
        uint outTokenIndex = 0; // EURe
        curveContract.exchange_underlying(
            inTokenIndex,
            outTokenIndex,
            wxdaiAmount,
            minReceive
        );
        uint256 eureReceived = IERC20(eureTokenAddress).balanceOf(
            address(this)
        );
        uint256 minimallyAcceptedEure = (wxdaiAmount / (oraclePrice / 1e15)) *
            curveMaxDiff;

        require(
            eureReceived > minimallyAcceptedEure,
            "EURe amount received lower than expected from the oracle price.
                Revert to prevent sandwiching attacks."
        );
    }
```

The function `price_oracle()` implemented by Curve uses the price of the 3xCRV LP token as an intermediary to calculate the price of EURe per USD:

```
def price_oracle() -> uint256:
    usd_tkn: uint256 = CurveCryptoSwap(POOL).price_oracle()  // @audit-info
        price of x3CRV LP token compared to EURe (EURe / x3CRV)
```

```
3       vprice: uint256 = StableSwap(BASE_POOL).get_virtual_price()  // @audit-info
            price of the LP token of x3CRV in USD
4       return vprice * 10**18 / usd_tkn  // @audit-info USD/x3CRV / (EURe / x3CRV)
            = USD/EURe
```

This means that `oraclePrice` is expressed in `USD`/`EURe` and not `wxDAI`/`EURe` which may differ if wxDAI is not close to the 1 USD value.

Impact

If the value of wxDAI varies widely from the value of 1 USD, the sandwich protection will incorrectly estimate the amount of EURe it should receive. This will allow for disadvantageous swaps for the user and prevent expected swaps.

My understanding is that the DAI peg to the dollar is not guaranteed in the long term which may break the use of this contract. See `MakerDAO forum`.

Code Snippets

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimActionUpgradeable.sol#L186-L212

https://gnosisscan.io/address/0xE3FFF29d4DC930EBb787FeCd49Ee5963DADf60b6#code

https://docs.curve.fi/cryptoswap-exchange/cryptoswap/pools/crypto-pool/#price_oracle

https://docs.curve.fi/cryptoswap-exchange/twocrypto-ng/pools/oracles/?h=get_virtual_price#get_virtual_price

Recommendation

Use an off-chain oracle to get the price of GNO in EURe to correctly protect the two swaps against sandwich attacks.

Response

This is fixed in commits `0x99e3aad` and `0x3bd9dab` by computing the price of GNO in EUR using Chainlink oracles.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

Enforce at least 1 day time threshold when setting config

The claim registry enforces that each validator's rewards will not be automatically claimed more than once every day through an if block in `getClaimableAddresses()`. It would be nice if `_setConfig()` disallow setting `timeThreshold` to below 1 day to avoid confusion for users trying to get their rewards claimed more than once a day.

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimRegistryUpgradeable.sol#L174

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimRegistryUpgradeable.sol#L385-L395

Response: updated in commit `0x7981485` to enforce `register()` and `updateConfig()` can only use time thresholds greater than a day.

Random public function comment

There is a comment about public functions in between two public functions in the claim registry that should be removed.

https://github.com/gnosischain/autoclaim-registry/blob/0fb58ff748f2b2d4cfd68a9c60333be136e2ed20/src/ClaimRegistryUpgradeable.sol#L197

Responde: the comment has been removed in commit `0x7981485`